Análisis de Algoritmos

Algoritmos y Estructuras de Datos

Rodrigo Pérez Rodríguez

Universidad Rey Juan Carlos

Contenido

1. Introducción
2. Método experimental
3. Modelos matemáticos
4. Modelo de coste
5. Clasificación de órdenes de crecimiento
6. Diseño de algoritmos más rápidos
7. Predicción de rendimiento
8. Obstáculos al analizar el rendimiento
9. Gestión de las dependencias de las entradas
10. Notación Big-Oh
11. Análisis del uso de memoria
12. Discusión

Introducción

Motivación: Ejemplos del mundo real

Analogías:

- **Aeronave:** ¿A qué altura puede volar? ¿Qué distancia alcanza con cierta cantidad de combustible?
- **Medicamento:** ¿Qué enfermedades cura? ¿Qué contraindicaciones tiene según la dosis?
- **Algoritmo:** ¿Cuánto tiempo tardará en ejecutarse? ¿Cuánta memoria necesita?

Objetivo:

- Predecir el rendimiento en función del tamaño de la entrada
- Optimizar el uso de recursos computacionales

Factores que afectan al tiempo de ejecución

Muchos factores influyen:

- Velocidad del microprocesador
- Datos concretos de entrada (distribución, ordenación, cantidad)
- Programa concreto que implementa el algoritmo
- Compilador o intérprete utilizado
- Sistema operativo
- Procesos ejecutándose simultáneamente

Enfoque:

- Nos centraremos en los factores **más importantes**
- Crearemos modelos simplificados pero útiles

Nuestro enfoque: Dos actividades principales

1. **Modelos de costes simplificados:**
   - Análisis matemático de la especificación del algoritmo
   - Aproximaciones que consideran solo factores importantes
   - No todos los detalles, solo los dominantes
2. **Validación experimental:**
   - Ejecutar programas que implementen los algoritmos
   - Obtener evidencia empírica
   - Comprobar que el modelo se aproxima a la realidad

Método experimental

El método científico aplicado a algoritmos

Proceso:

1. **Ejecutar** el programa y realizar mediciones precisas
2. **Proponer** como hipótesis un modelo matemático consistente con las mediciones
3. **Realizar predicciones** usando el modelo
4. **Verificar** si las predicciones son acertadas con nuevos experimentos
5. **Validar** o modificar el modelo hasta que sea consistente

Características importantes:

- Los experimentos deben ser **reproducibles**
- Las hipótesis deben ser **refutables**

Hipótesis refutables vs no refutables

× **Hipótesis NO refutable:**

"Este algoritmo es muy rápido"

- No está claro qué significa "muy rápido"
- No permite diseñar experimentos para refutarla

✓ **Hipótesis refutable:**

"Cuando duplicamos el número de valores de entrada, el tiempo de ejecución no crece más del doble"

- Específica y medible
- Permite diseñar experimentos concretos

Observaciones cualitativas

Observación básica (casi de perogrullo):

- Los algoritmos tardan más cuanto mayor es el número de datos a procesar
- Como lavar más platos lleva más tiempo

Pregunta interesante:

¿Cuánto se incrementa el tiempo cuando se duplican los datos?

Posibles respuestas:

- ¿El mismo tiempo? (constante)
- ¿Solo un poco más? (sublineal)
- ¿El doble? (lineal)
- ¿Mucho más del doble? (cuadrático, cúbico, exponencial...)

El problema ThreeSum

Problema:

- Dado un fichero con n enteros distintos
- Contar el número de tripletas cuya suma es cero
- Ejemplo: $-442472 + 155091 + 287381 = 0$

Preguntas:

- ¿Cuánto tardará con diferentes tamaños de fichero?
- ¿Cómo varía el tiempo en función del tamaño?

Ficheros de prueba:

- `1Kints.txt`, `2Kints.txt`, `4Kints.txt`, `8Kints.txt`...
- Tamaños crecientes para observar el patrón

Implementación: ThreeSum.java

```
public class ThreeSum {
    public static int count(int[] a) {
        int n = a.length;
        int count = 0;
        for (int i = 0; i < n; i++)
            for (int j = i+1; j < n; j++)
                for (int k = j+1; k < n; k++)
                    if (a[i] + a[j] + a[k] == 0)
                        count++;
        return count;
    }

    public static void main(String[] args) {
        int[] a = In.readInts(args[0]);
        System.out.println(count(a));
    }
}
```

Algoritmo de fuerza bruta:

- Tres bucles anidados
- Examina todas las tripletas posibles

Mediciones precisas: Stopwatch

API de Stopwatch:

Método	Descripción
`Stopwatch()`	crea un cronómetro
`double elapsedTime()`	tiempo desde que se creó

Implementación:

```
public class Stopwatch {
    private final long start;

    public Stopwatch() {
        start = System.currentTimeMillis();
    }

    public double elapsedTime() {
        long now = System.currentTimeMillis();
        return (now - start) / 1000.0;
    }
}
```

Cliente: DoublingTest

```
public class DoublingTest {
    public static double timeTrial(int n) {
        int[] a = new int[n];
        for (int i = 0; i < n; i++)
            a[i] = StdRandom.uniform(-1000000, 1000000);

        Stopwatch timer = new Stopwatch();
        int count = ThreeSum.count(a);
        return timer.elapsedTime();
    }

    public static void main(String[] args) {
        for (int n = 250; true; n += n) {
            double time = timeTrial(n);
            System.out.printf("%7d %7.1f\n", n, time);
        }
    }
}
```

Estrategia: Doblar el tamaño del array en cada iteración

Resultados experimentales

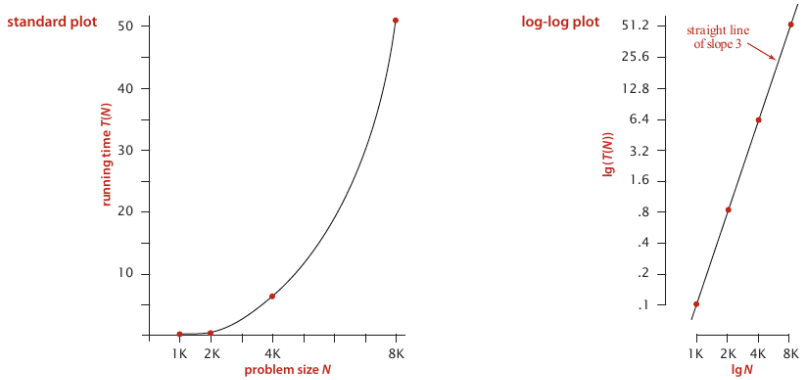
Salida de DoublingTest:

n	tiempo (seg)
250	0.0
500	0.0
1000	0.3
2000	0.4
4000	3.1
8000	25.2

Observaciones:

- El tiempo crece **rápidamente**
- No es proporcional al tamaño
- Mucho más que el doble cuando se duplica n

Gráfica estándar: n vs $T(n)$



Analysis of experimental data (the running time of `ThreeSum.count()`)

Observación:

- La curva crece muy rápidamente
- Difícil ver el patrón exacto en escala normal

Gráfica log-log: $\lg(n)$ vs $\lg(T(n))$

En la gráfica derecha anterior:

- Eje X: $\lg(n)$
- Eje Y: $\lg(T(n))$
- Aparece una **recta de pendiente 3**

Análisis matemático:

Ecuación general de una recta: $y = mx + b$ donde m es la pendiente y b es la ordenada en el origen.

Si $\lg(T(n))$ es una recta de pendiente 3:

$$\lg(T(n)) = 3\lg(n) + \lg(a)$$
$$\lg(T(n)) = \lg(n^3) + \lg(a)$$
$$\lg(T(n)) = \lg(an^3)$$
$$T(n) = an^3$$

Hipótesis: El tiempo de ejecución es cúbico

Calculando la constante a

Modelo propuesto: $T(n) = an^3$

Despejando a con datos experimentales:

$$a = \frac{T(n)}{n^3}$$

Para $n = 2000$ y $T(n) = 0{,}4$:

$$a = \frac{0{,}4}{2000^3} = 5 \times 10^{-11}$$

Ahora podemos predecir:

- Para $n = 8000$: $T(8000) = 5 \times 10^{-11} \times 8000^3 = 25{,}6$ seg
- Valor medido: 25.2 seg ✓
- Para $n = 16000$: $T(16000) = 204{,}8$ seg (predicho) vs 221.3 seg (medido) ✓

Ley potencial

Muchos fenómenos siguen una ley potencial:

$$y = ax^b$$

En nuestro caso:

- $y = T(n)$ (tiempo de ejecución)
- $x = n$ (tamaño de la entrada)
- $b = 3$ (exponente, determinado por el algoritmo)
- $a = 5 \times 10^{-11}$ (constante, dependiente de la máquina)

Validación:

- El modelo predice tiempos futuros con buena precisión
- Ganamos confianza en la hipótesis $T(n) = an^3$

Modelos matemáticos

Metodología de Knuth

Donald E. Knuth observó:

Se puede construir un modelo del tiempo de ejecución basado en dos factores:

1. **Coste de ejecución** de cada sentencia
   - Depende: microprocesador, compilador, SO, etc.
   - Fácil de medir experimentalmente
2. **Frecuencia de ejecución** de cada sentencia
   - Depende: del programa y de la entrada
   - El desafío está en determinar esta frecuencia

Tiempo total:

$$T = \sum_{\text{sentencias}} (\text{coste} \times \text{frecuencia})$$

Análisis de ThreeSum: Frecuencias

```
public static int count(int[] a) {
    int n = a.length;
    int count = 0;                              // 1 vez
    for (int i = 0; i < n; i++)                 // n veces
        for (int j = i+1; j < n; j++)           // n(n-1)/2 veces
            for (int k = j+1; k < n; k++)       // n(n-1)(n-2)/6 veces
                if (a[i] + a[j] + a[k] == 0)// n(n-1)(n-2)/6 veces
                    count++;                    // 0 a n(n-1)(n-2)/6 veces
    return count;
}
```

Pregunta clave: ¿Cuántas veces se ejecuta la expresión booleana del `if`?

Se evalúa una vez por cada tripleta de valores (i, j, k). Son **combinaciones de n elementos tomados de 3 en 3**.

Ejemplo con $n = 4$

```
for (int i = 0; i < n; i++)
  for (int j = i+1; j < n; j++)
    for (int k = j+1; k < n; k++)
      if (a[i] + a[j] + a[k] == 0)
        count++;
```

Con $n = 4$:

i	j	k
0	1	2
0	1	3
0	2	3
1	2	3

Total: 4 tripletas $= \frac{4 \times 3 \times 2}{6} = 4$
La expresión booleana del `if` se ejecuta **4 veces**

Ejemplo con $n = 5$

```
for (int i = 0; i < n; i++)
  for (int j = i+1; j < n; j++)
    for (int k = j+1; k < n; k++)
      if (a[i] + a[j] + a[k] == 0)
        count++;
```

Con $n = 5$:

i	j	k
0	1	2
0	1	3
0	1	4
0	2	3
0	2	4
0	3	4
1	2	3
1	2	4
1	3	4
2	3	4

Total: 10 tripletas $= \frac{5 \times 4 \times 3}{6} = 10$

Fórmula general

Para n elementos:
El número de tripletas (i, j, k) con $i < j < k$ es:

$$
\begin{aligned}
C_{n,3} = \binom{n}{3} &= \frac{n!}{3!(n-3)!} = \frac{n \cdot (n-1) \cdot (n-2) \cdot (n-3)!}{3! \cdot (n-3)!} \\
&= \frac{n(n-1)(n-2)(n-3)!}{3 \cdot 2 \cdot 1 \cdot (n-3)!} = \frac{n(n-1)(n-2)(n-3)!}{6(n-3)!} \\
&= \frac{n(n-1)(n-2)}{6}
\end{aligned}
$$

Conclusión:

- La expresión booleana del `if` se ejecuta $\frac{n(n-1)(n-2)}{6}$ veces
- El `count++` se ejecuta entre 0 y $\frac{n(n-1)(n-2)}{6}$ veces (depende de los datos)

Aproximación virgulilla ($\sim$)

Problema: Las expresiones matemáticas son largas
Para la expresión del `if`:

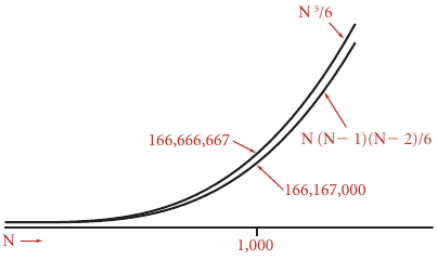
$$\frac{n(n-1)(n-2)}{6} = \frac{n^3}{6} - \frac{n^2}{2} + \frac{n}{3}$$

Para $n = 1000$:

- Primer término: $\frac{n^3}{6} \approx 166{,}666{,}667$
- Resto: $-\frac{n^2}{2} + \frac{n}{3} \approx -499{,}667$

Solución: Notación virgulilla

- Despreciamos términos que contribuyen poco
- $\frac{n^3}{6} - \frac{n^2}{2} + \frac{n}{3} \sim \frac{n^3}{6}$



Leading-term approximation

Definición formal de $\sim$

Definición:
Escribimos $g(n) \sim f(n)$ para indicar que:

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = 1$$

Ejemplo:

- $\frac{n^3}{6} - \frac{n^2}{2} + \frac{n}{3} \sim \frac{n^3}{6}$ porque $\lim_{n\to\infty} \frac{n^3/6 - n^2/2 + n/3}{n^3/6} = 1$

Orden de crecimiento:

- Si $g(n) \sim af(n)$ con $f(n) = n^b(\log n)^c$, decimos que $f(n)$ es el **orden de crecimiento** de $g(n)$ (ignoramos a)
- **¿Por qué esta forma?** $n^b(\log n)^c$ captura las complejidades típicas
- Ejemplos: n $(b = 1, c = 0)$, $n \log n$ $(b = 1, c = 1)$, n^2 $(b = 2, c = 0)$

Aproximaciones virgulilla típicas

Función	Aproximación $\sim$	Orden de crecimiento
$\frac{n^3}{6} - \frac{n^2}{2} + \frac{n}{3}$	$\frac{n^3}{6}$	n^3
$\frac{n^2}{2} - \frac{n}{2}$	$\frac{n^2}{2}$	n^2
$\lg n + 1$	$\lg n$	$\lg n$
$3n + 5$	$3n$	n
$n \lg n + n$	$n \lg n$	$n \lg n$

Observación sobre el logaritmo:

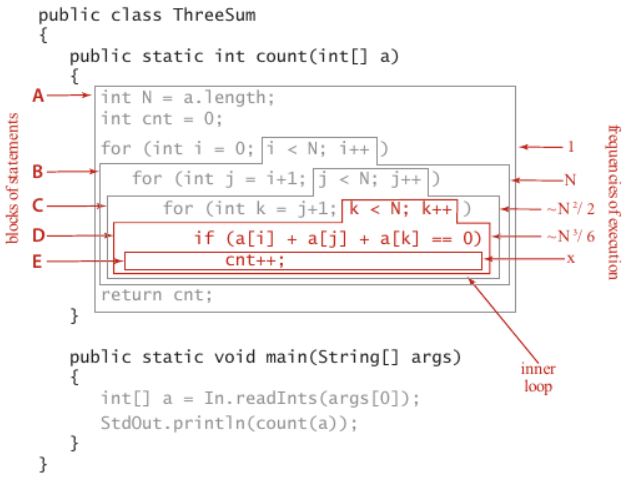
- No especificamos la base porque $\log_a n = \frac{\log_b n}{\log_b a}$
- Ejemplo: $\log_2 n = \frac{\log_{10} n}{\log_{10} 2} \approx 3{,}32 \cdot \log_{10} n$
- Solo difieren en una constante, que ignoramos en el orden de crecimiento

Órdenes de crecimiento comunes

De menor a mayor:

- 1 (constante)
- $\log n$ (logarítmico)
- n (lineal)
- $n \log n$ (linearítmico)
- n^2 (cuadrático)
- n^3 (cúbico)
- 2^n (exponencial)

Análisis completo de ThreeSum



Anatomy of a program's statement execution frequencies

Tabla de frecuencias y tiempos

Bloque	Tiempo	Frecuencia	Aproximación	Total
E	t_0	x (depende entrada)	—	$t_0 x$
D	t_1	$\binom{n}{3} = \frac{n(n-1)(n-2)}{6}$	$\sim \frac{n^3}{6}$	$\frac{t_1 n^3}{6}$
C	t_2	$\binom{n}{2} = \frac{n(n-1)}{2}$	$\sim \frac{n^2}{2}$	$\frac{t_2 n^2}{2}$
B	t_3	n	—	$t_3 n$
A	t_4	1	—	t_4

Tiempo total:

$$T(n) = \frac{t_1}{6} n^3 + \frac{t_2}{2} n^2 + t_3 n + t_4 + t_0 x$$

$$T(n) \sim \frac{t_1}{6} n^3 \text{ (si } x \text{ es pequeña)}$$

Orden de crecimiento: n^3

El bucle más interno

Observación clave:

El factor dominante suele ser el tiempo de ejecución de un pequeño grupo de sentencias

Bucle más interno:

- Las sentencias que se ejecutan con más frecuencia
- En ThreeSum: la evaluación del `if`
- Frecuencia: $\sim n^3/6$
- Determina el orden de crecimiento

Estrategia de análisis:

1. Identificar el bucle más interno
2. Calcular su frecuencia de ejecución
3. Eso determina el orden de crecimiento

Reflexión sobre ThreeSum

¿De qué depende el orden de crecimiento?

El orden de crecimiento NO depende de:

- El programa concreto
- El ordenador en que se ejecuta

El orden de crecimiento SÍ depende de:

- El **algoritmo** utilizado
- A veces, del modelo de entrada

Para ThreeSum: Depende de que examina todas las posibles tripletas

El algoritmo de fuerza bruta

En el caso de ThreeSum, el algoritmo es la fuerza bruta:

"Calcular la suma de todas las tripletas distintas contando aquellas cuya suma es cero"

Consecuencia:

- Cualquier implementación (en cualquier lenguaje)
- Ejecutada en cualquier ordenador
- Tendrá un tiempo de ejecución proporcional a n^3

Concepto clave: Separar el algoritmo de su implementación permite adquirir conocimientos aplicables a cualquier ordenador

Modelo de coste

¿Qué es un modelo de coste?

Definición:
Un modelo de coste define las **operaciones básicas** que usamos para medir el rendimiento del algoritmo.

Para ThreeSum:

- Contamos accesos al array
- No contamos operaciones aritméticas, comparaciones, etc.
- Simplifica el análisis

Ventaja:

- Afirmaciones matemáticas precisas sobre el algoritmo
- Independientes de la implementación concreta
- Permite analizar más alla del "tiempo de ejecución"

Proposición sobre ThreeSum

Proposición A:

El algoritmo de fuerza bruta 3-sum usa $\sim n^3/2$ accesos al array para calcular el número de tripletas de suma cero en n números

Demostración:

- Hay $\sim n^3/6$ tripletas
- Para cada tripleta se accede a 3 posiciones del array
- Total: $3 \times \frac{n^3}{6} = \frac{n^3}{2}$ accesos

Q.E.D. (Quod Erat Demonstrandum)

Relación modelo de coste - tiempo de ejecución

Objetivo:
Definir modelos de coste donde el orden de crecimiento del coste coincida con el orden de crecimiento del tiempo de ejecución.

Criterio:

- El modelo debe incluir operaciones del **bucle más interno**
- Son las sentencias más frecuentemente ejecutadas

Resultados:

- **Proposiciones:** sobre el modelo de coste (demostrables matemáticamente)
- **Propiedades:** sobre el tiempo de ejecución (requieren experimentos)

Importante: El tiempo real depende de hardware, compilador, etc., y **no se puede calcular matemáticamente**. El modelo de coste sí es analizable matemáticamente.

Resumen: Pasos para analizar un programa

Para muchos programas, el desarrollo de un modelo matemático del tiempo de ejecución conlleva estos pasos:

1. Desarrollar un modelo de las entradas
   - Incluye definición del tamaño del problema
2. Identificar el bucle más interno
3. Definir un modelo de coste
   - Incluye operaciones del bucle más interno
4. Determinar la frecuencia de ejecución
   - Puede requerir análisis matemático

Nota: Si hay múltiples métodos, consideramos cada uno por separado

Ejemplo: Análisis de Whitelist

Programa: Búsqueda binaria integrada en `Whitelist`

Análisis de `StaticSETofInts` (búsqueda binaria):

- **Modelo de las entradas:** array `a[]` de tamaño n
- **Bucle más interno:** sentencias en el bloque `while`
- **Modelo de coste:** comparaciones de dos posiciones del array
- **Análisis:** número de comparaciones $\leq \lg n + 1$

Ejemplo: Análisis de Whitelist (cont.)

Análisis de `Whitelist`:

- **Modelo de las entradas:** n números en whitelist y m números en entrada estándar (suponemos $m \gg n$)
- **Bucle más interno:** sentencias en el bucle `while`
- **Modelo de coste:** operación de comparar (heredada de `BinarySearch`)
- **Análisis:** teniendo en cuenta el de `BinarySearch`: $m(\lg n + 1)$

Conclusión: Orden de crecimiento de `Whitelist` es $m \lg n$

Whitelist: Consideraciones importantes

Consideraciones sobre el análisis:

- Si n es pequeño, el coste de entrada/salida dominaría
- El número de comparaciones depende de la entrada: entre $\sim m$ y $\sim m \lg n$
  - Depende de cuántos números de entrada están en whitelist
  - Típicamente $\sim m \lg n$
- Suponemos que `Arrays.sort()` es pequeño comparado con $m \lg n$
  - `Arrays.sort()` implementa mergesort
  - Orden de crecimiento: $n \lg n$
  - Como $m \gg n$, entonces $m \lg n$ domina a $n \lg n$

Whitelist: Impacto de duplicar entradas

El modelo sustenta que la búsqueda binaria permite ejecución en tiempo razonable cuando m y n son grandes:

Si duplicamos m (números en entrada):

- $m' = 2m$
- El tiempo se incrementa al doble: $\sim 2m \lg n$

Si duplicamos n (números en whitelist):

- $n' = 2n$
- El tiempo NO se duplica
- Se incrementa en proporción a $\lg(2n)$
- Incremento mucho menor que duplicar

Clasificación de órdenes de crecimiento

Órdenes de crecimiento habituales

Descripción	**Función**	**Estructura típica**	**Ejemplo**
constante	1	sentencia simple	sumar dos números
logarítmico	$\lg n$	dividir por la mitad	búsqueda binaria
lineal	n	bucle simple	encontrar el máximo
linearítmico	$n \lg n$	dividir y conquistar	mergesort
cuadrático	n^2	bucle doble anidado	comprobar todos los pares
cúbico	n^3	bucle triple anidado	comprobar todas las tripletas
exponencial	2^n	búsqueda exhaustiva	todos los subconjuntos

Nota: Existen otros órdenes como $n^2 \lg n$ o $n^{3/2}$, pero estos son los más habituales

Descripción de cada orden de crecimiento

Constante (1):

- Una sola sentencia o número fijo de sentencias
- Ejemplo: asignar un valor, devolver un campo

Logarítmico ($\log n$):

- Divide el problema a la mitad en cada paso
- Ejemplo: búsqueda binaria

Lineal (n):

- Un bucle que recorre n elementos
- Ejemplo: encontrar el máximo en un array

Órdenes de crecimiento (continuación)

Linearítmico ($n \log n$):

- Divide y vencerás
- Ejemplo: mergesort, quicksort (caso medio)

Cuadrático (n^2):

- Dos bucles anidados
- Ejemplo: comparar todos los pares

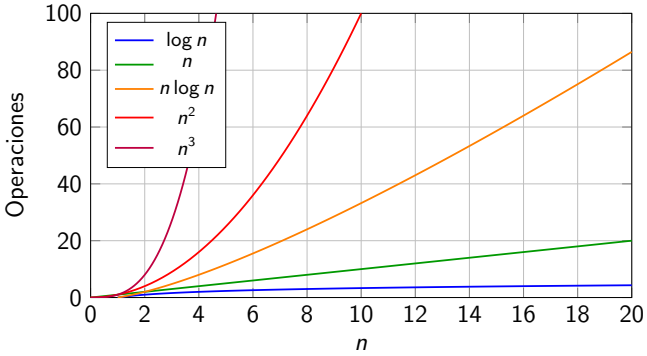
Cúbico (n^3):

- Tres bucles anidados
- Ejemplo: ThreeSum

Exponencial (2^n):

- Explora todas las combinaciones
- Ejemplo: subconjuntos, problema del viajante (fuerza bruta)

Comparación gráfica



Observaciones:

- Cuadrático y cúbico **no escalan** para problemas grandes
- Logarítmico, lineal y linearítmico son **viables**
- Exponencial es **impracticable** incluso para n pequeños

Diseño de algoritmos más rápidos

Motivación: ¿Podemos hacerlo mejor?

Pregunta:
¿Podemos usar mergesort ($n \log n$) y búsqueda binaria ($\log n$) para mejorar ThreeSum?

Estrategia:

1. Empezar con un problema más simple: TwoSum
2. Encontrar una solución más rápida
3. Aplicar la misma idea a ThreeSum

Problema TwoSum

Definición:

- Contar pares de suma cero en un array de n enteros distintos

Solución de fuerza bruta:

- Dos bucles anidados
- Orden de crecimiento: n^2 (cuadrático)

¿Podemos mejorar?

- Sí, usando ordenación y búsqueda binaria

Solución mejorada: TwoSumFast

```java
import java.util.Arrays;

public class TwoSumFast {
    public static int count(int[] a) {
        Arrays.sort(a);  // mergesort: ~ n log n
        int n = a.length;
        int count = 0;
        for (int i = 0; i < n; i++) {
            // Buscar -a[i] con búsqueda binaria: ~ log n
            int j = Arrays.binarySearch(a, -a[i]);
            if (j > i)  // evitar contar dos veces
                count++;
        }
        return count;
    }
}
```

Análisis:

- Ordenación: $\sim n \log n$
- n búsquedas binarias: $\sim n \log n$
- **Total:** $\sim n \log n$ (linearítmico) ✓

Observación clave para TwoSum

Idea:

`a[i]` forma parte de un par de suma cero

$$\Updownarrow$$

`-a[i]` está en el array

Ventaja de ordenar:

- Búsqueda binaria es logarítmica
- Mucho más rápida que búsqueda lineal
- El coste de ordenar se amortiza con n búsquedas

Aplicando la idea a ThreeSum

```java
import java.util.Arrays;

public class ThreeSumFast {
    public static int count(int[] a) {
        // Ordenar el array: O(n log n)
        Arrays.sort(a);
        int n = a.length;
        int count = 0;

        // Para cada par (i, j) con i < j
        for (int i = 0; i < n; i++)
            for (int j = i+1; j < n; j++) {
                // Buscar si existe k tal que a[i]+a[j]+a[k]=0
                // Es decir, buscar -(a[i]+a[j]) en el array
                int k = Arrays.binarySearch(a, -(a[i] + a[j]));
                // Solo contar si k > j para evitar duplicados
                if (k > j) count++;
            }
        return count;
    }
}
```

Análisis de ThreeSumFast

Costes:

- Ordenación: $\sim n \log n$
- Bucles anidados: $n(n-1)/2 \sim n^2/2$ iteraciones
- Búsqueda binaria en cada iteración: $\sim \log n$
- Total búsquedas: $\sim \frac{n^2}{2} \log n$

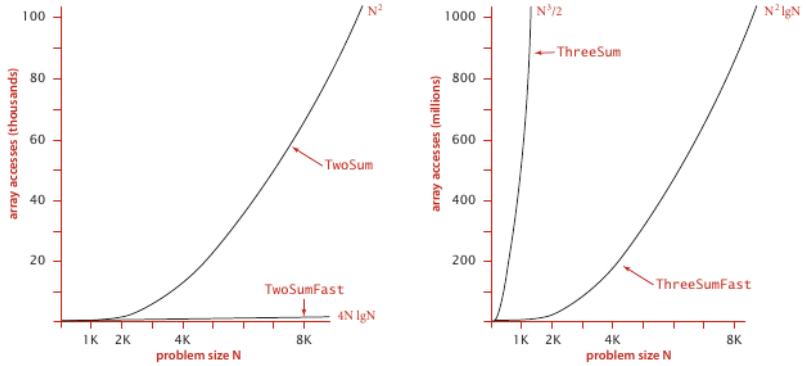
Orden de crecimiento:

- $n \log n + n^2 \log n \sim n^2 \log n$
- El término dominante es $n^2 \log n$
- El coste de ordenar es insignificante

Mejora:

- De n^3 (cúbico) a $n^2 \log n$
- ¡Mucho más rápido para n grandes!

Comparación de rendimiento



Costs of algorithms to solve the 2-sum and 3-sum problems

Tabla resumen: 2-Sum y 3-Sum

Problema	Fuerza bruta	Algoritmo mejorado
2-Sum	n^2	$n \log n$
3-Sum	n^3	$n^2 \log n$

Preguntas abiertas:

- ¿Existe un algoritmo **lineal** para 2-Sum? NO
- ¿Existe un algoritmo **linearítmico** para 3-Sum? NO SE SABE
- Se cree que el mejor es **cuadrático**

Cotas inferiores

Definición:
Una **cota inferior** es un límite en el mejor orden de crecimiento posible para *cualquier* algoritmo que resuelva un problema.

Importancia:

- Nos dice cuándo **parar de buscar** mejores algoritmos
- Si alcanzamos la cota inferior, el algoritmo es **óptimo**

Dificultad:

- Es muy difícil establecer cotas inferiores no triviales
- Requiere probar que *ningún* algoritmo puede hacerlo mejor

Predicción de rendimiento

Técnica de doblar la entrada

Método alternativo muy útil:

1. Ejecutar con entradas que **doblan el tamaño**
2. Observar los **tiempos de ejecución**
3. Calcular el **ratio** entre tiempos consecutivos
4. Esperar a que el ratio **converja a** 2^b

Si el ratio converge a 2^b**:**

- El orden de crecimiento es $\sim n^b$
- Podemos **predecir** tiempos futuros

Implementación: DoublingRatio

```
public class DoublingRatio {
    public static double timeTrial(int n) {
        int[] a = new int[n];
        for (int i = 0; i < n; i++)
            a[i] = StdRandom.uniform(-1000000, 1000000);
        Stopwatch timer = new Stopwatch();
        int count = ThreeSum.count(a);
        return timer.elapsedTime();
    }

    public static void main(String[] args) {
        double prev = timeTrial(125);
        for (int n = 250; true; n += n) {
            double time = timeTrial(n);
            System.out.printf("%6d %7.1f %5.1f\n", n, time, time/prev);
            prev = time;
        }
    }
}
```

Ejemplo de ejecución: ThreeSum

Salida:

n	tiempo	ratio
250	0.0	2.7
500	0.0	3.6
1000	0.1	1.9
2000	0.4	7.2
4000	3.1	7.9
8000	25.2	8.1

Análisis:

- El ratio converge a $\sim 8 = 2^3$
- Por tanto, $b = 3$
- Orden de crecimiento: n^3 ✓

¿Por qué funciona este método?

Pregunta clave:
¿Por qué el ratio se aproxima a una constante cuando se duplica n?

Respuesta:
Esto ocurre con todos los órdenes de crecimiento habituales (excepto el exponencial)

Para entenderlo necesitamos:

- Definir $T(n)$: el tiempo de ejecución del algoritmo para entrada de tamaño n
- Asumir que $T(n)$ tiene la forma general $T(n) \sim an^b \log n$
- Donde a es una constante, b es el exponente que buscamos

¿Qué es $T(n)$?

Definición de $T(n)$:
$T(n)$ representa el tiempo de ejecución del algoritmo cuando el tamaño de entrada es n

Para ThreeSum:

- $T(250)$ es el tiempo cuando $n = 250$ ($\sim$ 0,0 seg)
- $T(500)$ es el tiempo cuando $n = 500$ ($\sim$ 0,0 seg)
- $T(8000)$ es el tiempo cuando $n = 8000$ ($\sim$ 25,2 seg)

Forma genérica:
Asumimos $T(n) \sim an^b \log n$ donde:

- a = constante que depende del hardware, compilador, etc.
- b = exponente que caracteriza el orden de crecimiento
- Es la forma típica de casi todos los algoritmos habituales

Proposición B (ratio de doblado)

Proposición B (ratio de doblado):
Si $T(n) \sim an^b \log n$, entonces $T(2n)/T(n) \sim 2^b$

¿Qué significa?
Si duplicamos el tamaño de entrada (de n a $2n$), el ratio de tiempos converge a 2^b

Importancia:
Nos permite determinar b (el exponente del orden de crecimiento) simplemente observando el ratio de tiempos cuando duplicamos n

Demostración de la Proposición B

Partimos de: $T(n) \sim an^b \log n$

Paso 1: Sustituir $T(2n)$ y $T(n)$

$$\frac{T(2n)}{T(n)} = \frac{a(2n)^b \log(2n)}{an^b \log n}$$

Paso 2: Simplificar $(2n)^b = 2^b \cdot n^b$ y $\log(2n) = \log 2 + \log n$

$$= \frac{a \cdot 2^b \cdot n^b \cdot (\log 2 + \log n)}{a \cdot n^b \cdot \log n}$$

Demostración de la Proposición B (cont.)

Paso 3: Cancelar a y n^b

$$\frac{T(2n)}{T(n)} = \frac{2^b(\log 2 + \log n)}{\log n}$$

Paso 4: Separar la fracción

$$\begin{aligned} &= 2^b \left(\frac{\log 2 + \log n}{\log n} \right) \\ &= 2^b \left(\frac{\log 2}{\log n} + \frac{\log n}{\log n} \right) \\ &= 2^b \left(\frac{\log 2}{\log n} + 1 \right) \end{aligned}$$

Paso 5: Cuando $n \to \infty$, el término $\frac{\log 2}{\log n} \to 0$

$$\sim 2^b \cdot 1 = 2^b \quad \text{Q.E.D.}$$

Observación importante sobre el factor logarítmico

Paradoja aparente:

- En el modelo matemático: $T(n) = an^b \log n$ (el $\log n$ **importa**)
- En el ratio: $T(2n)/T(n) \sim 2^b$ (el $\log n$ **desaparece**)

¿Por qué desaparece en el ratio?
En el límite, el término $\frac{\log 2}{\log n} \to 0$, por lo que el factor logarítmico se cancela en el ratio

Consecuencia práctica:

- El ratio 2^b NO nos dice si hay un factor $\log n$ o no
- Ejemplo: tanto n como $n \log n$ dan ratio ~ 2 al duplicar
- Por eso linearítmico puede confundirse con lineal en experimentos
- Es una técnica rápida para estimar el orden de crecimiento

¿Y si el orden de crecimiento NO es potencial?

Alcance de la Proposición B:

La fórmula $T(n) \sim an^b \log n$ incluye el factor $\log n$, por tanto SÍ funciona para:

- Constante, lineal, linearítmico ($n \log n$), cuadrático, cúbico...
- Cualquier orden de la forma n^b o $n^b \log n$

¿Qué pasa con el exponencial?

Si $T(n) \sim 2^n$, el ratio NO converge a una constante:

$$\frac{T(2n)}{T(n)} \sim \frac{2^{2n}}{2^n}$$
$$\sim 2^{2n-n} = 2^n$$

(crece exponencialmente sin converger)

Conclusión:

- Este método NO funciona para algoritmos exponenciales
- Para exponenciales, el ratio sigue creciendo sin converger

Ventajas de este método

Muy simple:

- No requiere hacer gráficas log-log
- No requiere calcular logaritmos
- Solo observar cuándo converge el ratio

Predicciones:

- Para $n' = 2n$: $T(n') = T(n) \times 2^b$
- Para otros tamaños: ajustar el ratio

Aplicaciones:

- Estimar orden de crecimiento rápidamente
- Descubrir problemas de rendimiento
- Validar optimizaciones

Ejemplo: Algoritmo linearítmico ($n \log n$)

Algoritmo: Mergesort (ordenación)

n	tiempo (seg)	ratio
1000	0.001	—
2000	0.002	2.0
4000	0.004	2.0
8000	0.008	2.0
16000	0.017	2.1

Análisis:

- Ratio $\sim 2 = 2^1 \Rightarrow b = 1$
- **Problema:** El ratio NO distingue entre n y $n \log n$ (ambos dan ~ 2)
- **Solución:** Análisis matemático del código confirma que es $n \log n$

Estimación de viabilidad

Pregunta clave al programar:

¿Podrá este programa procesar estos datos en tiempo razonable?

Tabla de predicción (tiempo de ejecución):

Orden de crecimiento	**Función**	**Tiempo para tamaño** n	$10n$	$100n$	**Factor** $(10\times)$
logarítmico	$\lg n$	segundos	segundos	segundos	1.3
lineal	n	segundos	segundos	minutos	10
linearítmico	$n \lg n$	segundos	minutos	horas	13
cuadrático	n^2	segundos	horas	semanas	100
cúbico	n^3	segundos	semanas	siglos	1,000
exponencial	2^n	segundos	nunca	nunca	∞

Factor: cuánto se multiplica el tiempo al multiplicar el tamaño por 10

Análisis de la tabla

Si un programa tarda horas en ejecutarse:

- **Lineal:** Multiplicar entrada por 10 $\Rightarrow$ tiempo $\times$ 10
- **Linearítmico:** $\times$ 10 $\Rightarrow$ tiempo $\times$ 13 (aprox)
- **Cuadrático:** $\times$ 10 $\Rightarrow$ tiempo $\times$ 100 (!)
- **Cúbico:** $\times$ 10 $\Rightarrow$ tiempo $\times$ 1000 (!!)

Conclusión:

- Algoritmos cuadráticos y cúbicos **no escalan**
- Para problemas grandes, necesitamos algoritmos con mejor orden de crecimiento

La ley de Moore (histórica)

Ley de Moore:

Cada 18 meses se duplicaba la velocidad y memoria de los ordenadores

Implicación:

- Cada 5 años: ordenadores $\sim$ 10 veces más rápidos

Estado actual:

- Ya **no está vigente** (límites físicos)
- Aún más importante entender el orden de crecimiento

Observación:

- Algoritmo lineal o linearítmico: **sigue** la ley de Moore
- Algoritmo cuadrático o cúbico: **no puede** seguirla

Obstáculos al analizar el rendimiento

Obstáculos al analizar el rendimiento

Algunas de las **suposiciones** realizadas para formular las hipótesis sobre el rendimiento podrían ser **falsas**.

En ese caso se pueden:

- Analizar más detalles
- Hacer otras suposiciones
- Formular nuevas hipótesis

Veamos los principales obstáculos que podemos encontrar...

Obstáculo 1: Constantes grandes

¿Podemos ignorar los coeficientes de términos de menor grado?

- Si c es pequeño: $2n^2 + cn \sim 2n^2$ ✓
- Si c es 10^3 o 10^6: la aproximación no tiene sentido

Conclusión: Hay que vigilar si las constantes son grandes.

Obstáculo 2: El bucle más interno no domina

Dos situaciones problemáticas:

1. El **modelo de coste** podría no haber tenido en cuenta el verdadero bucle más interno
2. El **tamaño de la entrada** podría ser pequeño, haciendo que el coste del bucle más interno identificado no sea dominante

Solución: Refinar el modelo de coste.

Obstáculo 3: Tiempo de ejecución variable

La suposición de que cada instrucción **siempre tarda lo mismo** en ejecutarse no siempre está justificada.

Ejemplo: Hardware de caché de memoria RAM

- Arrays pequeños (caben en caché): acceso muy rápido
- Arrays grandes (no caben en caché): acceso más lento

Experimento con ThreeSum:

- Tras converger a ratio ≈ 8 (cúbico)
- El ratio sube a valores más altos debido a este fenómeno

Obstáculo 4: Consideraciones del sistema

En el sistema hay **muchos parámetros** que pueden afectar el rendimiento:

- La propia **JVM** tiene muchas opciones y controles
- **Recolector de basura**
- Compilador **Just in Time** (JIT): compila bytecode a código máquina durante la ejecución, optimizando el código más usado
- Procesos simultáneos (descargas, actualizaciones, etc.)

Consecuencia: Los experimentos no son todo lo reproducibles que deberían.

Obstáculo 5: Dependencia fuerte de las entradas

Suposición habitual: El rendimiento no depende de la entrada específica, sino de su tamaño.

Contraejemplo: Variante de 3-Sum que comprueba si existe alguna tripleta de suma 0

- Si la primera tripleta es de suma 0: tiempo **constante**
- Si no hay ninguna tripleta de suma 0: tiempo **cúbico**

Cuando no podemos justificar esta suposición:

- Podemos obtener resultados inconsistentes
- Podemos ser incapaces de validar las hipótesis

Obstáculo 6: Múltiples parámetros

¿Qué ocurre si no hay un solo número o una sola entrada?

Ejemplo: Construir una estructura de datos y realizar operaciones sobre ella

- Tamaño de la estructura de datos
- Número de operaciones
- Ambos son parámetros del problema

Caso Whitelist:

- n números en whitelist
- m números a buscar
- Tiempo de ejecución: $\sim m \lg n$

Conclusión sobre los obstáculos

A pesar de todos estos obstáculos, entender el orden de crecimiento del tiempo de ejecución es un conocimiento valioso.

Las técnicas y métodos que hemos aprendido son potentes y aplicables a muchos problemas.

Responsabilidad del/de la programador/a

Al igual que:

- Ingeniería aeroespacial: ¿aterrizará en el mar o en una ciudad?
- Medicina: ¿curará o matará al/a la paciente?

Un/a programador/a debe poder responder: ¿tardará 1 segundo o 1 año?

El orden de crecimiento ayuda a responder a esta pregunta.

Gestión de las dependencias de las entradas

El problema de la dependencia de las entradas

Hemos visto que la variante del problema 3-Sum que comprueba si existe alguna tripleta de suma cero **depende fuertemente de las entradas**.

El orden de crecimiento puede variar entre **constante** y **cúbico**.

¿Cómo podemos realizar un análisis más detallado en casos como este?

Veremos varias estrategias...

Estrategia 1: Modelos de la entrada

Podríamos **asumir** que la entrada está compuesta por números aleatorios.

Problema: Esta suposición podría no ser realista.

Ejemplo: Procesamiento del genoma

- El objetivo es descubrir un modelo de la entrada
- Es imposible modelar las entradas porque se desconoce su distribución
- ¿Cómo predecir lo que tardará en procesar el genoma de otras personas?

Estrategia 2: Garantías de rendimiento en el peor caso

Algunas aplicaciones requieren determinar una **cota superior del tiempo de ejecución** independientemente de la entrada.

Sistemas de tiempo real crítico:

- Máquinas para radiar a pacientes
- Control de una central nuclear
- Software de aviónica o robótica
- ABS de un coche

Si el programa no realiza su tarea en un tiempo acotado, **pase lo que pase**, las consecuencias pueden ser catastróficas.

Garantías y seguridad

Incluso en sistemas **no críticos**, si no proporcionamos cotas del tiempo de ejecución o memoria consumida:

$\Rightarrow$ Podemos ser víctimas de **ataques de denegación de servicio**

Es importante diseñar algoritmos y estructuras de datos con garantías.

Proposición C

En la implementación de Bag con listas enlazadas (y de Stack y Queue), **todas las operaciones tardan un tiempo constante en el peor de los casos**.

Prueba: Inmediata del código, siempre que Java tarde tiempo constante en crear un nodo.

Estrategia 3: Algoritmos aleatorios

Una forma importante de introducir garantías de rendimiento es mediante la **aleatorización del algoritmo**.

Ejemplo 1: Quicksort

- Peor caso: cuadrático $\mathcal{O}(n^2)$
- Entrada aleatoria: linearítmico $\mathcal{O}(n \lg n)$

Ejemplo 2: Hashing para tablas de símbolos

- Peor caso: lineal $\mathcal{O}(n)$
- Con aleatoriedad garantizada: constante $\mathcal{O}(1)$

Estas garantías **no son absolutas**, pero la probabilidad de que no se cumplan es despreciable $\Rightarrow$ se utilizan tanto como las del caso peor.

Estrategia 4: Secuencias de operaciones

Si la entrada son operaciones, el **orden en el que se ejecutan** puede influir.

Ejemplo con pila:

- $n/2$ push y luego $n/2$ pop
- Alternando push y pop

¿Por qué importa la secuencia?

- Con **arrays dinámicos**: los push pueden causar redimensionamientos costosos
- $n/2$ push seguidos $\Rightarrow$ varios redimensionamientos
- Alternando $\Rightarrow$ menos redimensionamientos

El análisis debe considerar ambas opciones o incluir un modelo razonable de la secuencia.

Estrategia 5: Análisis amortizado

Otra forma de proporcionar **garantías de rendimiento** es amortizar el coste.

Idea: Repartir el coste dispar de las operaciones entre todas ellas, independientemente de su contribución individual.

Dar una garantía basada en la **media del coste de todas las operaciones**.

Ejemplo típico: Pila implementada con un array de tamaño cambiante

- Algunas operaciones son constantes
- Otras son lineales (redimensionar)
- ¿Cuál es el coste promedio?

Recordatorio: Pila con array redimensionable

En el tema 2 vimos la implementación `ResizingArrayStack`.

Problema a resolver:

- Array de capacidad fija: desperdicia memoria o se llena
- **Solución:** Redimensionar el array dinámicamente

Estrategia:

- `push()`: Si el array está lleno, duplicar su tamaño
- `pop()`: Si el array está 1/4 lleno, reducir a la mitad

¿Por qué 1/4 *y no* 1/2*? Para evitar thrashing (redimensionar constantemente).*

Método resize()

```
private void resize(int max)
{
    // Mover pila a un nuevo array temp, de tamaño max
    Item[] temp = (Item[]) new Object[max];
    for (int i = 0; i < n; i++)
        temp[i] = a[i];
    a = temp;
}
```

Coste: Proporcional a n (número de elementos a copiar)

push() con redimensionamiento

```
public void push(Item item)
{
    // Añadir item a la cima de la pila
    if (n == a.length)
        resize(2*a.length);
    a[n++] = item;
}
```

Coste:

- Caso normal: constante (un acceso al array)
- Caso redimensionamiento: lineal $\sim n$

pop() con redimensionamiento

```
public Item pop()
{
    // Extraer item de la cima de la pila
    Item item = a[--n];
    a[n] = null;  // Evitar loitering
    if (n > 0 && n == a.length/4)
        resize(a.length/2);
    return item;
}
```

Nota: Reducimos cuando está 1/4 lleno, no 1/2 lleno.
Esto evita el **thrashing**: añadir/quitar elementos no redimensiona constantemente.

Análisis del número de accesos al array

Supongamos que n (tamaño del array) es potencia de 2.

¿Cuántos accesos al array se realizan si se hacen n llamadas consecutivas a `push()`?

$$\begin{aligned}\text{Total} &= n + (4 + 8 + 16 + \ldots + 2n) \\ &= n + 4(1 + 2 + 4 + \ldots + n/2) \\ &= n + 4(n - 1) \\ &= 5n - 4\end{aligned}$$

Desglose:

- n: los n push normales (un acceso cada uno)
- Resto: redimensionamientos

Explicación del cálculo de redimensionamientos

Para redimensionar de tamaño k a $2k$, necesitamos copiar k elementos.

Coste de copiar k elementos: $2k$ accesos

- k accesos para **leer** del array viejo
- k accesos para **escribir** en el array nuevo
- Total: $2k$ accesos al array

Los redimensionamientos ocurren:

$$2 \to 4 \text{ (coste 4)}, \quad 4 \to 8 \text{ (coste 8)}, \quad \ldots, \quad n \to 2n \text{ (coste } 2n\text{)}$$

Coste total de redimensionamientos:

$4 + 8 + 16 + \ldots + 2n = 4(1 + 2 + 4 + \ldots + n/2) = 4n - 4$

Total de accesos

Total de accesos para n push consecutivos: $n + (4n - 4) = 5n - 4$

- n: accesos normales de push
- $4n - 4$: accesos por redimensionamientos

Observación importante:

- La **mayoría** de operaciones push son constantes (1 acceso)
- Algunas operaciones push **causan redimensionamiento** y son lineales
- En particular, el último push (de n a $2n$) cuesta $2n$ accesos
- Pero si **distribuimos** ese coste entre todas las operaciones, $\frac{5n-4}{n} \approx 5$ accesos por operación

$\Rightarrow$ El coste **medio** (amortizado) es constante, aunque algunas operaciones individuales sean lineales.

Veamos una prueba más formal de este resultado...

Análisis amortizado: definición

Coste amortizado

Repartir el coste de las operaciones costosas entre todas las operaciones de una secuencia.

Características:

- Proporciona garantía del **peor caso**
- Para **cualquier secuencia de operaciones**
- Comenzando por una estructura de datos vacía

Conclusión para push():

- Última operación puede ser lineal
- Pero el **promedio** es constante
- Coste amortizado: $\mathcal{O}(1)$

Análisis amortizado: Proposición

Proposición D

En la implementación de Stack con arrays que cambian su tamaño, el número medio de accesos al array **para cualquier secuencia de** `push` **y** `pop`, comenzando con una estructura de datos vacía, es **constante en el peor de los casos**.

Es decir: **coste amortizado constante** $\mathcal{O}(1)$ por operación.

Prueba de la Proposición E

El array crece de n a $2n$:

Situación: El array acaba de crecer a tamaño n (contenía $n/2$ elementos, se hizo push del elemento $n/2 + 1$ que causó el crecimiento)

Operaciones push: Para que crezca de nuevo, entran $n/2 - 1$ elementos más (hasta llenar el array con n elementos). El siguiente push causará el crecimiento.

Coste de crecer a 2n: $4n$ accesos

- $2n$ para inicializar el array de tamaño $2n$
- $2n$ para copiar n ítems (*temp[i]* y *a[i]*)

Total: $4n + (n/2 - 1)$ accesos

- para $n/2 - 1$ push

Cálculo del coste medio

Promediamos entre las $n/2 - 1$ operaciones push:

$$\frac{4n + (n/2 - 1)}{n/2 - 1} = \frac{4n}{n/2 - 1} + 1$$

$$\approx \frac{4n}{n/2} + 1 \quad \text{(para } n \text{ grande)}$$

$$= 8 + 1 = 9$$

$\Rightarrow$ Cada operación push tiene un **coste amortizado de 9 accesos** al array.

$\Rightarrow$ Coste amortizado constante $\mathcal{O}(1)$ por operación

Conclusión: Aunque una operación push puede ser $\mathcal{O}(n)$, el coste **promedio** por operación es constante.

Análisis para push y pop combinados

Nota: Hacer el análisis para cualquier secuencia de `push` y `pop` es más difícil.

Pero se puede demostrar que también es constante amortizado.

Aplicabilidad:

- Este tipo de análisis es aplicable a muchas aplicaciones
- Los arrays redimensionables se usan en muchas estructuras de datos y algoritmos

Objetivo: Algoritmos implementados con código compacto y claro que proporcionen:

- Garantías de rendimiento
- Buen rendimiento para valores de entrada que nos interesen

Resumen: Análisis amortizado

Ventajas:

- Permite analizar estructuras con operaciones de coste variable
- Proporciona garantías del peor caso para secuencias
- Coste amortizado constante para `ResizingArrayStack`

Comparación con lista enlazada:

- Array redimensionable: coste constante **amortizado**
- Lista enlazada: coste constante en el **peor caso**

Trade-off:

- Array: mejor localidad de caché, acceso por índice
- Lista enlazada: sin operaciones costosas individuales

Notación Big-Oh

Definición de Big-Oh

Notación $\mathcal{O}$ (Big-Oh):

$$f(n) \in \mathcal{O}(g(n)) \text{ si existen constantes } c \text{ y } n_0 \text{ tales que}$$
$$|f(n)| \leq c|g(n)| \text{ para todo } n > n_0$$

Interpretación:

- $g(n)$ es una **cota superior asintótica** de $f(n)$
- $f(n)$ crece **como mucho** tan rápido como $g(n)$

Ejemplo:

- $3n^2 + 5n + 2 \in \mathcal{O}(n^2)$
- También: $3n^2 + 5n + 2 \in \mathcal{O}(n^3)$ (pero menos preciso)

Limitaciones de Big-Oh

Problema:
Big-Oh es una **cota superior**, el rendimiento real podría ser mucho mejor.

Ejemplo:

- Un algoritmo puede ser tanto $\mathcal{O}(n^2)$ como $\sim an \log n$
- La notación Big-Oh es **menos precisa**

Utilidad de Big-Oh:

- ✓ Facilita desarrollo de cotas superiores
- ✓ Proporciona **garantías** sobre el rendimiento
- × No es ideal para **predicciones** exactas
- × No permite comparar algoritmos con precisión

Big-Omega y Big-Theta

Big-Omega (Ω):
$f(n) \in \Omega(g(n))$ si existen c y n_0 tales que $|f(n)| \geq c|g(n)|$ para $n > n_0$

- **Cota inferior** asintótica
- $f(n)$ crece **al menos** tan rápido como $g(n)$

Big-Theta (Θ):
$f(n) \in \Theta(g(n))$ si $f(n) \in \mathcal{O}(g(n))$ y $f(n) \in \Omega(g(n))$

- **Equivalencia asintótica**
- $f(n)$ y $g(n)$ crecen **al mismo ritmo**
- Algoritmo **óptimo**: no existe otro con mejor Θ

Analogía con relaciones:

- $\mathcal{O}$ como $\leq$, Ω como $\geq$, Θ como $=$

Nuestro enfoque en este curso

Preferimos afirmaciones precisas:

- "ThreeSum usa $\sim n^3/2$ accesos al array"
- "El número de veces que se ejecuta el `if` es $\sim n^3/6$ en el peor caso"

En lugar de:

- "El tiempo de ejecución de ThreeSum es $\mathcal{O}(n^3)$"

Razones:

- Las afirmaciones precisas dan **más información**
- Permiten **mejores predicciones**
- Son viables para los algoritmos que estudiaremos

Ejercicio: ¿Cuál es la complejidad?

Analicemos el siguiente código:

```
public class ThreeSumBool {
    public static boolean count(int[] a) {
        int n = a.length;
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i+1; j < n; j++) {
                for (int k = j+1; k < n; k++) {
                    if (a[i] + a[j] + a[k] == 0) {
                        return true;
                    }
                }
            }
        }
        return false;
    }
}
```

Solución: Complejidad de ThreeSumBool

Análisis:

- Tres bucles anidados sobre el array de tamaño n
- En el peor caso, no encuentra ningún triplete que sume 0
- Debe revisar todas las combinaciones posibles

Complejidad temporal:

$$\mathcal{O}(n^3)$$

Observación:

- En el mejor caso podría ser $\mathcal{O}(1)$ si encuentra un triplete en la primera iteración
- Pero el análisis del peor caso es $\mathcal{O}(n^3)$

Análisis del uso de memoria

¿Por qué analizar la memoria?

Motivación:

- No solo importa el **tiempo**
- También la **memoria** disponible limita la viabilidad
- Saber cuánta memoria necesita un algoritmo

Características en Java:

- El uso de memoria es **determinista**
- Mismo programa, mismo ordenador $\Rightarrow$ misma memoria
- Puede variar entre diferentes JVM o arquitecturas

Ventaja:

- Más fácil que analizar el tiempo
- Solo miramos las **declaraciones de variables**
- El resto del código no influye

Tamaños típicos de tipos primitivos

Valores típicos (arquitectura 64 bits):

Tipo	Bytes
`boolean`	1
`byte`	1
`char`	2
`int`	4
`float`	4
`long`	8
`double`	8
referencia	8

Nota sobre referencias:

- Una **palabra** es la unidad básica de datos que procesa el CPU
- En arquitectura 64 bits: 1 palabra $=$ 8 bytes $\Rightarrow$ referencias de 8 bytes
- En arquitectura 32 bits: 1 palabra $=$ 4 bytes $\Rightarrow$ referencias de 4 bytes
- Las referencias ocupan exactamente una palabra de memoria

Sobrecoste de objetos

Cada objeto Java tiene un sobrecoste de 16 bytes:

- Referencia a la clase del objeto
- Información del recolector de basura
- Información de sincronización para threads

Padding (relleno):

- Los objetos ocupan múltiplos del tamaño de palabra (8 bytes en 64 bits)
- Se añaden bytes de relleno al final si es necesario

Ejemplo: Integer

- Sobrecoste: 16 bytes
- Campo `int`: 4 bytes
- Subtotal: $16 + 4 = 20$ bytes
- Padding: 4 bytes (para completar hasta el siguiente múltiplo de 8: 24)
- **Total: 24 bytes** (¡6 veces el tamaño del `int`!)

Ejemplos de memoria ocupada por objetos

Date (3 campos int):

- Sobrecoste: 16 bytes
- $3 \times$ `int`: $3 \times 4 = 12$ bytes
- Padding: 4 bytes
- **Total: 32 bytes**

Counter (String + int):

- Sobrecoste: 16 bytes
- Referencia `String`: 8 bytes
- Campo `int`: 4 bytes
- Padding: 4 bytes
- **Total: 32 bytes** (más el objeto String)

Listas enlazadas: clase Node

Clase anidada (inner class) no estática:

- Sobrecoste adicional de **8 bytes**
- Referencia a la instancia de la clase contenedora

Nodo de lista enlazada:

- Sobrecoste objeto: 16 bytes
- Referencia a `Item`: 8 bytes
- Referencia a `Node`: 8 bytes
- Sobrecoste clase anidada: 8 bytes
- **Total: 40 bytes**

Pila de n Integers con lista enlazada:

- Stack (objeto contenedor): 32 bytes
  - 16 sobrecoste + 8 ref (`Node first`) + 4 int (`n`) + 4 padding
- n nodos: $40n$ bytes (40 bytes/nodo)
- n Integers: $24n$ bytes (24 bytes/Integer)
- **Total:** $32 + 40n + 24n = 32 + 64n$ **bytes**

Arrays: costes

Arrays como objetos:

- Sobrecoste: 16 bytes
- Longitud del array: 4 bytes
- Padding: 4 bytes
- Elementos: depende del tipo

Array de primitivos:

- `int[n]`: $24 + 4n$ bytes ($+$ padding si necesario)
- `double[n]`: $24 + 8n$ bytes

Array de objetos:

- `Date[n]`: $24 + 8n$ (referencias) $+$ $32n$ (objetos Date) $= 24 + 40n$ bytes

Arrays 2D: desglose del cálculo

Un array 2D en Java es un array de arrays

Ejemplo: `double[m][n]`

- **Array principal:** array de m referencias
  - Sobrecoste + longitud + padding: 24 bytes
  - m referencias (a los subarrays): $8m$ bytes
  - **Subtotal:** $24 + 8m$ bytes
- m **subarrays:** cada uno de n doubles
  - Cada subarray: $24 + 8n$ bytes
  - m subarrays: $m(24 + 8n) = 24m + 8mn$ bytes

Total: $(24 + 8m) + (24m + 8mn) = 24 + 8m + 24m + 8mn = 24 + 32m + 8mn$ bytes

Para m, n grandes: $\sim 8mn$ bytes (dominan los elementos)

String: Java 7+

Variables de instancia:

- Referencia a `char[] value`: 8 bytes
- `int hash`: 4 bytes (código hash precalculado)

String de longitud n:

- Objeto String: 32 bytes (16 sobrecoste + 8 ref + 4 int + 4 padding)
- Array de char: $24 + 2n$ bytes
- **Total:** $56 + 2n$ **bytes**

Substring en Java 7+:

- Crea un **nuevo array** de char
- Tiempo: lineal en la longitud de la subcadena
- Espacio: lineal en la longitud de la subcadena

Discusión

Preguntas para discusión

- ¿Por qué es importante el orden de crecimiento?
- ¿Cuándo usar análisis experimental vs matemático?
- ¿Qué significa que un algoritmo sea óptimo?
- ¿Cómo afecta la memoria a la viabilidad de un algoritmo?

¿Por qué es importante el orden de crecimiento?

Permite predecir el rendimiento:

- Estimar tiempos de ejecución para entradas grandes
- Comparar algoritmos de forma independiente del hardware
- Identificar cuellos de botella

Guía el diseño de software:

- Decidir qué algoritmos son viables
- Priorizar optimizaciones
- Planificar recursos computacionales
- ¡Diferencia entre 1 segundo y 1 año!

¿Cuándo usar análisis experimental vs matemático?

Análisis experimental:

- Validar modelos matemáticos
- Descubrir comportamiento en casos reales
- Comparar implementaciones concretas
- Identificar efectos del sistema (cache hits/misses, localidad de acceso; SO, scheduling, etc.)

Análisis matemático:

- Establecer garantías teóricas
- Comparar algoritmos independientemente
- Probar cotas superiores e inferiores
- Diseñar algoritmos óptimos

Conclusión

Ambos enfoques son complementarios y necesarios

¿Qué significa que un algoritmo sea óptimo?

Definición:

- Alcanza la cota inferior del problema
- No existe otro algoritmo con mejor orden de crecimiento
- Se expresa con notación Θ

Ejemplos:

- Mergesort: $\Theta(n \log n)$ comparaciones (óptimo para ordenación basada en comparaciones)
- Búsqueda binaria: $\Theta(\log n)$ comparaciones (óptima en arrays ordenados)

Importancia:

- Saber cuándo dejar de buscar mejores algoritmos
- Enfocarse en otros aspectos (claridad, memoria, constantes)

¿Cómo afecta la memoria a la viabilidad?

Restricciones de memoria:

- Memoria limitada puede hacer inviable un algoritmo rápido
- Trade-offs tiempo-espacio son comunes
- Ejemplo: tablas de lookup vs cálculo
  - **Lookup:** precalcular y almacenar resultados $\Rightarrow$ rápido pero usa memoria
  - **Cálculo:** calcular cuando se necesita $\Rightarrow$ ahorra memoria pero más lento

Consideraciones prácticas:

- Arrays: contiguos, buena localidad
- Listas enlazadas: dispersas, más memoria
- Estructuras auxiliares: mejoran tiempo pero usan espacio
- Cache y memoria virtual afectan rendimiento real

Conclusión

Analizar tiempo y espacio es esencial para evaluar viabilidad